Header

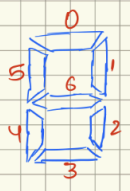
[redacted]

Lab Number: 4
Lab Name: High-Level Verilog

Q1.

Item Name	UPC	Discounted?	Expensive?
<New Item 1> Silver	0 0 0	No	Yes
<New Item 2> DEO	0 0 1	No	No
<New Item 3> FAn	0 1 1	Yes	No
<New Item 4> GOLD	1 0 0	No	Yes
<New Item 5> TEE	1 0 1	Yes	Yes
<New Item 6> HAT	1 1 0	Yes	No

(SiL)



S = 0, 5, 6, 2, 3
i = 4
L = 5, 4, 3
D = 0, 1, 2, 3, 4, 5
E = 0, 5, 6, 4, 3
O = ALL of em

F = 0, 5, 6, 4
A = 4, 5, 0, 1, 2
N = 4, 6, 2
G = 0, 5, 4, 3, 2, 6
T = 0, 1, 2
H = 5, 4, 6, 1, 2

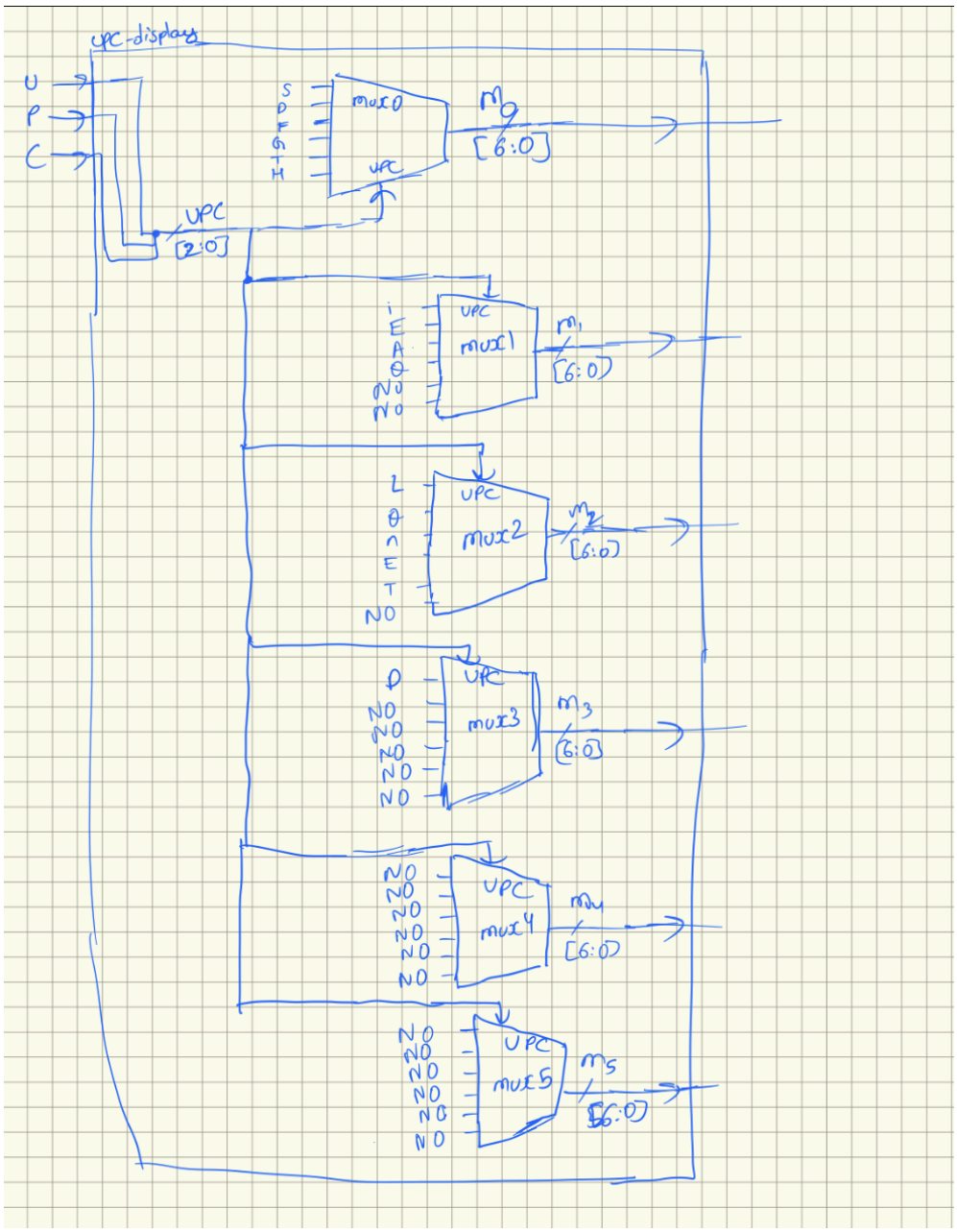
S = 7'b0010010
0 = 7'b1000000
F = 7'b0001110
G = 7'b0000010
T = 7'b1111000
H = 7'b0001001
i = 7'b1101111
E = 7'b0000110
A = 7'b0001000
O = 7'b0000000

NO = 7'b1111111
L = 7'b1000111
n = 7'b0101011

The above screenshot shows my version of the UPC code table with associated discounted and expensive tags for each product.
It also shows my work for determining the LED displays I wanted to activate for each UPC code product on the 7-seg displays.

The lower-third shows my finalized variables that represent the actual binary representations of all the letters and variables that will be used by upc_display module to drive the 7-seg displays.

Q2.

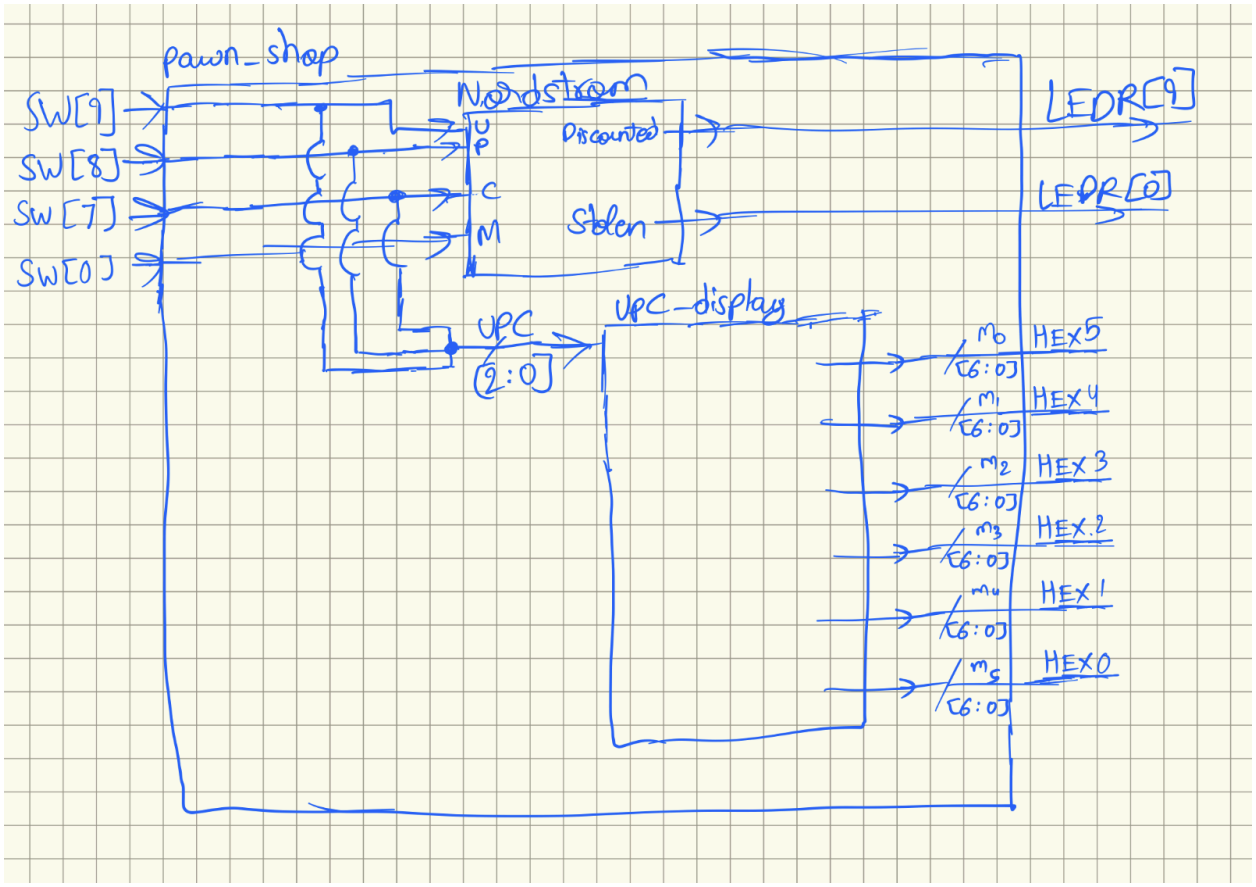


Here is the drawing of my upc_display circuit that drives the 7-seg displays. It takes signals U, P, C as inputs that encode the UPC code of an item and outputs the name of the product on the 7-segment displays.

Each mux drives its own 7-seg display, so the 6 inputs into the 6-to-1 mux represent 6 possible characters possible for those displays.

For instance, mux0 drives the first 7-seg display from the left in my configuration. So, the output of the first mux will be one of the 6 possible first characters of the 6 items from the table. Similarly, the output of the second mux will be the second character of items from the UPC table and so on…
The last 2 muxes are always empty (display nothing) because each of my item's names is only 3-4 characters long.

Q3.



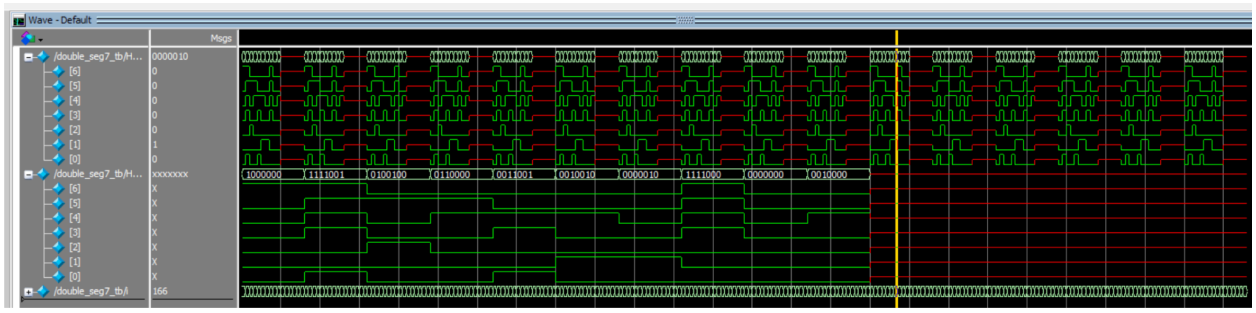
The above screenshot shows my pawn_shop module. It contains the Nordstrom module made in the previous lab and the aforementioned upc_display module. This module connects directly to the hardware, so its inputs/outputs are actual ports on the DE1-SoC device.
The inputs to this module are the switches we designated to represent the UPC code and Mark indicator. This module outputs to chosen LEDs that represent whether a product is discounted and or stolen as well as the actual name of the product through the 7-seg displays

This diagram shows the Nordstrom and upc_display module working side-by-side in the top-level pawn_shop module. They achieve different objectives, but both work towards being functional according to the specifications - one shows characteristics about a returned product and the other its actual name.

As hinted towards before by the example, the mux numbering is slightly different from the actual displays they drive (it's the reverse) - this is merely an artifact of the way the displays are laid out in the board, but the actual logic remains the same.

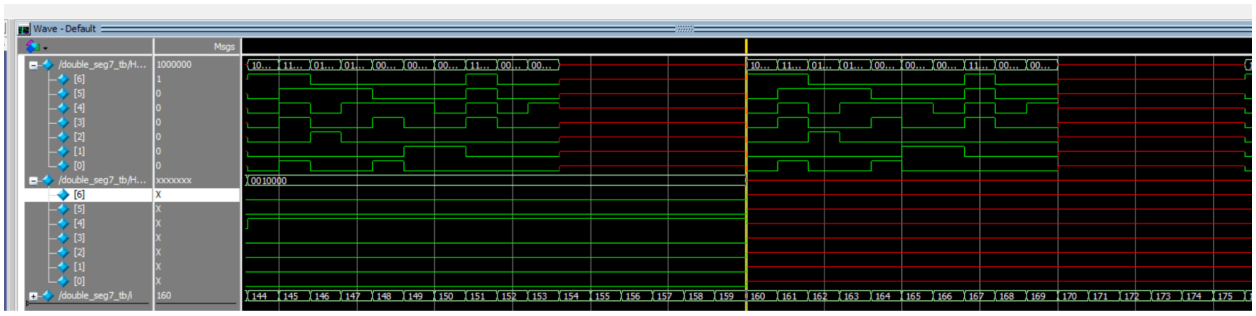
Q4.

Overall Screenshot:



This is the overall screenshot showing behavior for all possible switch configurations. We see that there is a point after which the second 7-seg display stops being known by the simulation. Zooming in reveals something interesting

Middle Portion:



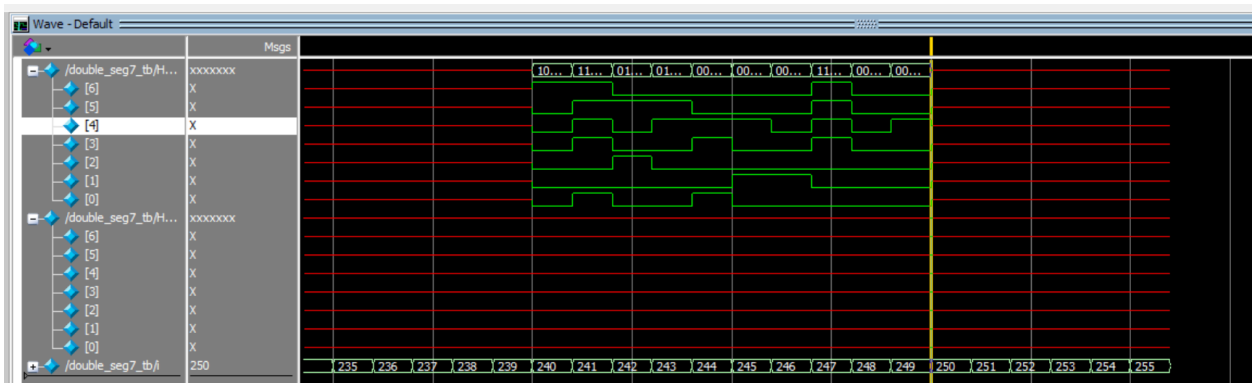
This screenshot more clearly shows the cliff the point after which second hex display becomes unknown – specifically at i = 160.

First and second hex display refer to the displays from the right.

A characteristic of the mapping code in this program is that the displays are only configured to properly display numbers 0-9 inclusive; the remaining non-accounted for possibilities result in unknown values for those switch configurations – this explains the red bars in modelsim which indicate that the values are unknown for certain regions. This also explains why the second hex displays stops being known past 159. In binary, 159 = 1001 1111.
For the second hex display, this is the last representable value since 1001 binary = 9. 160 in binary = 1010 0000 – the hex display 2 portion of this number is not representable by the hex display according to the given mapping, its and any values after 160 are all unrepresentable reliably by the second display.

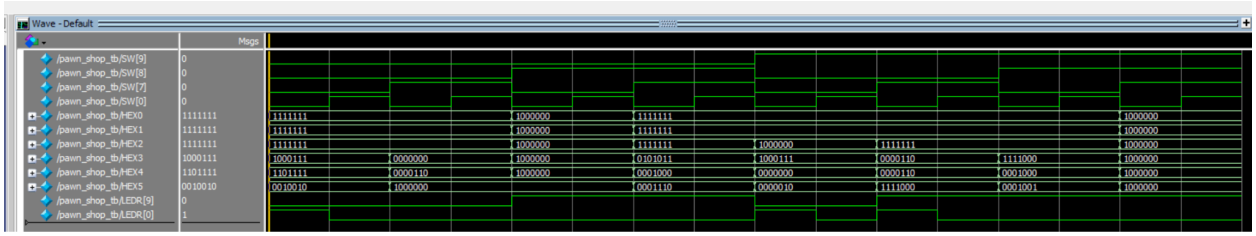
The first hex display cycles between representable and unrepresentable since it goes from 0000 to 1111 cyclically whilst the values until the end where its value is not known as well.

End portion:



Q5.

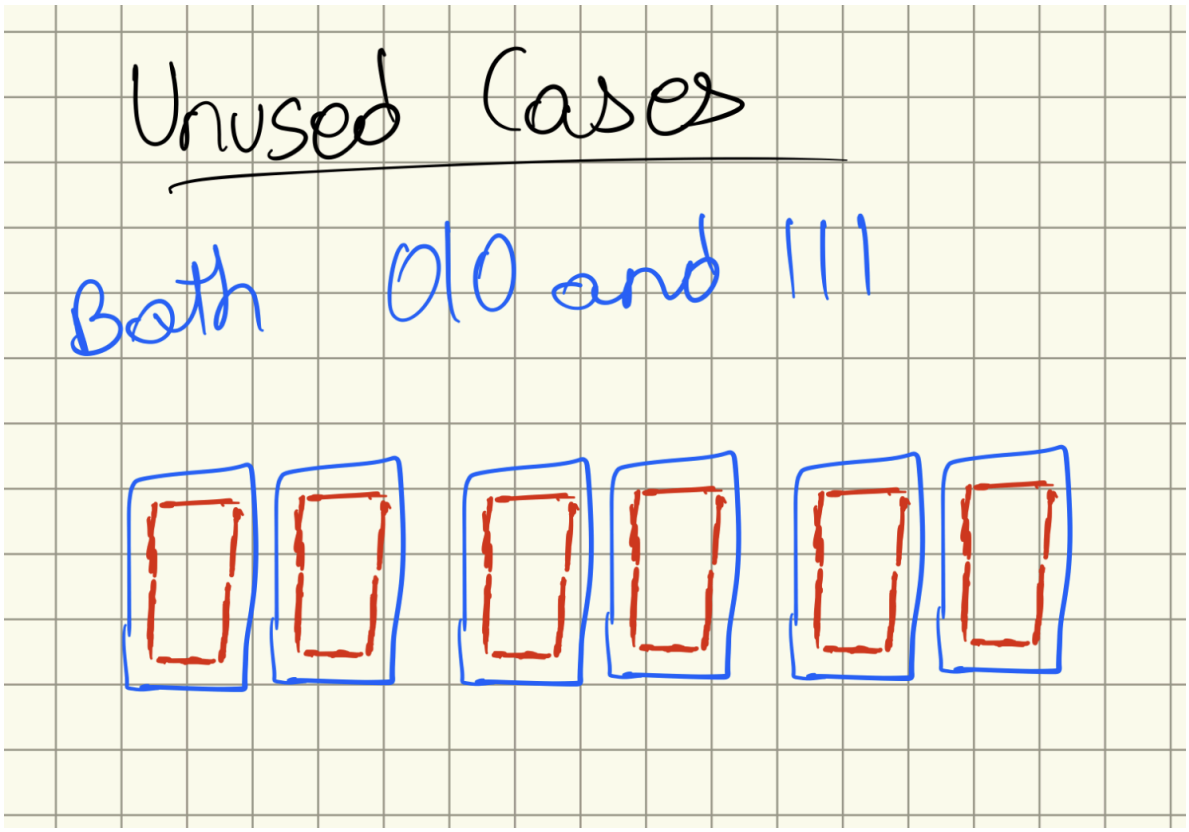
Overall Circuit:



The above screenshot shows overall behavior of the circuit given certain inputs. Since there are only 8 possible UPC code (6 used) and 2 possibilities for mark, the simulation is relatively small.
As we can see , there are only two cases where all the HEX values are 1000000 – which represents the default case. If we follow the modelsim, we see that only happening when SW[9] – SW[7] are the following: 010 and 111. This shows that the default case of all 0s (or Ds technically in my display mapping) is being activated for unused UPC codes. The rest of the UPC codes are being mapped correctly as shown by the HEX signals if associated with the respective switch inputs.

Due to the handling of the default case, there is no knowns, so there aren't red lines everywhere!

Q6.



Time Spent: 6.5 hours